Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, Douwe Kiela

Facebook AI Research; University College London; New York University

arXiv:2005.11401v1 | NeurIPS 2020

Abstract

Large pre-trained language models have been shown to store factual knowledge in their parameters, and achieve state-of-the-art results when fine-tuned on downstream NLP tasks. However, their ability to access and precisely manipulate knowledge is still limited, and hence on knowledge-intensive tasks their performance lags behind task-specific architectures. Additionally, providing provenance for their decisions and updating their world knowledge remain open research problems. We introduce retrieval-augmented generation (RAG), a general-purpose fine-tuning recipe that combines pre-trained parametric and non-parametric memory for language generation. We introduce RAG models where the parametric memory is a pre-trained seq2seq model and the non-parametric memory is a dense vector index of Wikipedia, accessed with a pre-trained neural retriever. We compare two formulations, one which conditions on the same retrieved passages for the whole generated sequence (RAG-Sequence) and one which can use different passages per token (RAG-Token). On three open-domain QA tasks, our models set the state of the art, outperforming parametric seq2seq models and task-specific retrieve-and-extract architectures. On Natural Questions, RAG achieves 44.5 exact match, using a DPR retriever and a FiD-style generator with the top 5 retrieved documents.

Keywords: retrieval-augmented generation, dense retrieval, question answering, language generation

1 Introduction

Knowledge-intensive tasks such as open-domain question answering require a model to access and reason over a large body of world knowledge. Pre-trained language models store a surprising amount of factual knowledge in their parameters, but this knowledge is difficult to update, difficult to inspect, and often imprecise. Retrieval-augmented models address these problems by combining a parametric generator with a non-parametric memory that can be searched at inference time.

We introduce RAG, a hybrid model that conditions a seq2seq generator on documents retrieved from a dense index of Wikipedia. The retriever is a dual-encoder model trained with a maximum inner product search objective, and the generator is a pre-trained encoder-decoder model. Because the index is external, the model's knowledge can be updated by replacing the index, and the retrieved documents provide provenance for the generated answer. Figure 1 shows the overall architecture.

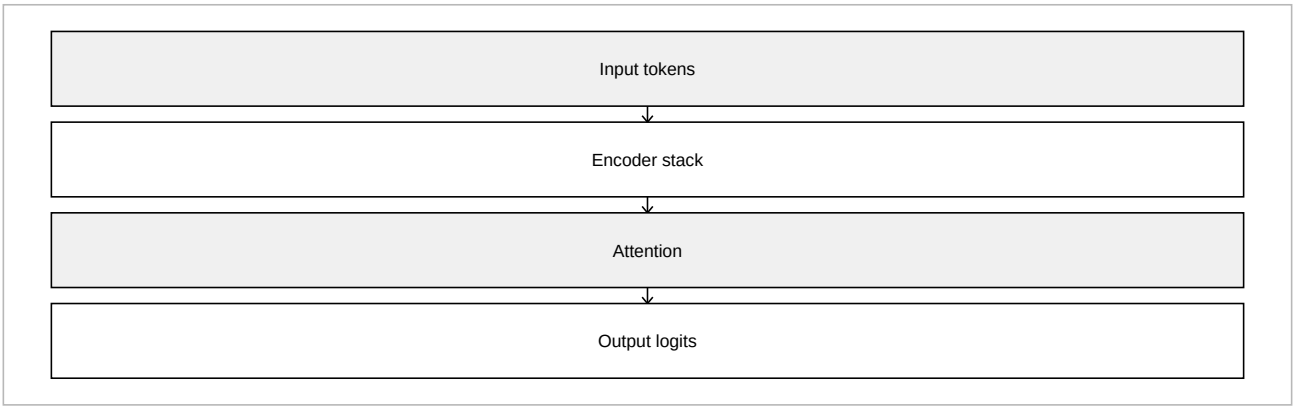

Figure 1: Overview of RAG: a query encoder retrieves passages from a dense index, and a generator conditions on the query and the retrieved passages.

2 Methods

RAG uses a question encoder to embed the input and a document encoder to embed each candidate document, and it retrieves the top-k documents by inner product. The generator then conditions on the question and the retrieved documents. We consider two ways of combining the retrieved evidence, RAG-Sequence and RAG-Token, described below.

2.1 Retrieval Model

The retriever is a dense dual encoder: a BERT-based question encoder maps the question to a vector, and a BERT-based document encoder maps each document to a vector in the same space. We use a pre-trained DPR retriever and keep it fixed during the first stage of training. The retrieval distribution is computed with a softmax over inner products, as in Equation (1), and the index contains 21 million Wikipedia passages.

$$p(z \mid x) = \mathrm{softmax}(d(z)^\top q(x)) \tag{1}$$

2.2 Generator

The generator is a pre-trained sequence-to-sequence model. We use a FiD generator, which encodes the question together with each retrieved document independently and then concatenates the representations before decoding. This design scales to many retrieved documents because the encoder cost grows linearly rather than quadratically in the number of passages. We use the top 5 retrieved documents in all main experiments.

2.3 RAG-Sequence and RAG-Token

In RAG-Sequence the model retrieves a set of documents once and uses the same set to generate the entire output sequence. In RAG-Token the model can attend to different documents at each decoding step, which allows it to combine evidence from several passages. Both variants are trained by maximizing the marginal likelihood over the retrieved documents, as in Equation (2). RAG-Token is more expressive, while RAG-Sequence is simpler and faster at inference time.

$$p(y \mid x) = \sum_{z \in \text{top-k}} p(z \mid x)\, p(y \mid x, z) \tag{2}$$

3 Experiments

We evaluate RAG on open-domain question answering and on abstractive question answering. The datasets are Natural Questions, TriviaQA, WebQuestions, CuratedTrec, and MSMARCO. All models use the same Wikipedia index of 21 million passages, and we report exact match for QA and ROUGE-L for generation.

3.1 Datasets

Natural Questions contains questions written by real users and answers annotated in Wikipedia. TriviaQA contains trivia questions with evidence documents. WebQuestions and CuratedTrec are smaller, older benchmarks, and MSMARCO contains questions derived from search queries. For training we use the training splits of Natural Questions and TriviaQA, and we evaluate on the development and test splits of all five datasets.

3.2 Results

On Natural Questions, RAG achieves 44.5 exact match, which is state of the art among models that do not use task-specific retrieval heuristics. On TriviaQA, RAG achieves 56.8 exact match, and on WebQuestions it achieves 45.2. These results outperform both parametric seq2seq models and retrieve-and-extract pipelines. Table 1 reports the exact match scores for all five datasets.

We also find that RAG generates more specific and diverse answers than a closed-book model, and that the retrieved documents provide useful provenance. In a human evaluation, annotators preferred RAG outputs over closed-book outputs for factuality and specificity. The gains are largest on questions whose answers are rare in the training data, which is consistent with the hypothesis that retrieval supplies knowledge that the parameters do not store.

Table 1: Exact match on open-domain QA test sets. All RAG models use the top 5 retrieved documents.

Dataset	RAG-Sequence	RAG-Token	Closed-book
Natural Questions	44.5	44.1	32.5
TriviaQA	56.8	55.9	47.9
WebQuestions	45.2	44.8	38.1
CuratedTrec	52.0	51.4	43.0
MSMARCO	41.5	40.8	34.6

3.3 Ablations

We ablate the number of retrieved documents, the choice of retriever, and the training schedule. Table 2 shows that increasing the number of documents from 1 to 5 improves exact match, while further increases give diminishing returns. Freezing the retriever during the first stage and fine-tuning it later gives the best results, and training the retriever from scratch degrades performance by more than 2 points.

Table 2: Ablation of the number of retrieved documents on Natural Questions.

Top-k documents	Exact match
1	40.1
3	43.0
5	44.5
10	44.9

4 Related Work

Retrieval-augmented models have a long history in open-domain QA. Early systems used sparse retrieval such as TF-IDF or BM25 followed by a reader, while more recent systems use dense retrievers trained end-to-end. Our work differs from these systems in that the generator is a pre-trained seq2seq model that is fine-tuned jointly with the retriever, and in that the model can generate free-form answers rather than extract spans.

Our retriever builds on DPR, which uses a BERT-based dual encoder, and our generator builds on FiD, which fuses retrieved passages in the decoder. RAG combines these ideas and shows that joint training of retrieval and generation improves both components.

5 Conclusion

We have presented RAG, a general-purpose retrieval-augmented generation model that combines parametric and non-parametric memory. RAG achieves state-of-the-art results on open-domain QA and produces answers with provenance. We believe that retrieval-augmented models are a promising direction for knowledge-intensive NLP, and we release our code and index to support further research.

References

[1] Devlin et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL, 2019.

[2] Karpukhin et al. Dense Passage Retrieval for Open-Domain Question Answering. EMNLP, 2020.

[3] Izacard and Grave. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. EACL, 2021.

[4] Lewis et al. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. ACL, 2020.

[5] Robertson and Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 2009.

[6] Vaswani et al. Attention Is All You Need. NeurIPS, 2017.

[7] Kwiatkowski et al. Natural Questions: A Benchmark for Question Answering Research. TACL, 2019.

[8] Joshi et al. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. ACL, 2017.

[9] Guu et al. REALM: Retrieval-Augmented Language Model Pre-Training. ICML, 2020.

[10] Petroni et al. Language Models as Knowledge Bases? EMNLP, 2019.

A Appendix: Training Details

We train RAG on 8 NVIDIA V100 32GB GPUs with a batch size of 8 and a learning rate of 1e-4 for the generator and 1e-5 for the retriever. The first stage freezes the retriever and trains the generator for 10,000 steps; the second stage fine-tunes both components for 2,000 steps.

At inference time we use beam search for RAG-Sequence.